

Programmierung

–

Level

1.3

1.	Vorwort zu den Programmierübungen	3
2.	Ziel der Übung.....	3
3.	Was ist auf der ATC-Platine bereits vorbereitet?.....	3
4.	Neue Befehle/Konzepte in dieser Übung	4
5.	Erklärung der verwendeten Befehle	4
6.	Schritt-für-Schritt: Schreibe den Code selbst	6
5.1.	Schritt 1 – Pins festlegen	6
5.2.	Schritt 2 – setup(): Ein- und Ausgänge konfigurieren	6
5.3.	Schritt 3 – Aufgabe A: Taster steuert LED direkt	7
5.4.	Schritt 4 – Aufgabe B: Taster als Schaltbefehl	7
5.5.	Schritt 5 – Aufgabe C: Umschaltlogik	7
5.6.	Schritt 6 – Blinkmuster starten und stoppen (millis())	8
7.	Challenge	11
6.1.	Challenge A – Blinkrhythmus.....	11
6.2.	Challenge B – Geschwindigkeit verändern und LEDs ausschalten	11
8.	Typische Fehler & schnelle Diagnose	12
9.	Referenzlösung Aufgabe A	13
10.	Referenzlösung Aufgabe B.....	14
11.	Referenzlösung Aufgabe C.....	15
12.	Referenzlösung Bonus	16
13.	Referenzlösung Challenge A	18
14.	Referenzlösung Challenge B.....	20
15.	Abschluss	23

1. Vorwort zu den Programmierübungen

Die folgenden Programmierübungen sind so aufgebaut, dass sie Schritt für Schritt in die Arbeit mit Mikrocontrollern einführen.

Alle Übungen können mit dem Hardware-Einsteigerpaket Level 1 umgesetzt werden und bauen inhaltlich aufeinander auf.

Zu Beginn werden grundlegende Konzepte vermittelt und einfache Aufgaben umgesetzt. Mit jeder weiteren Übung werden neue Funktionen ergänzt und bestehende Inhalte erweitert.

Ziel ist es, ein solides Verständnis für den Aufbau von Programmen, den Umgang mit der Arduino IDE und das Zusammenspiel von Hard- und Software zu entwickeln.

Die Übungen sind bewusst übersichtlich gehalten und sollen dazu ermutigen, eigene Anpassungen vorzunehmen und mit dem Code zu experimentieren.

Es wird empfohlen, die Programmierübungen in der vorgesehenen Reihenfolge durchzuführen, da spätere Aufgaben auf den vorherigen aufbauen.

So entsteht nach und nach ein nachvollziehbarer Lernpfad, der als Grundlage für weiterführende Projekte dient.

2. Ziel der Übung

In dieser Programmierübung lernst du, zwei Taster als digitale Eingänge zu verwenden und damit die LEDs auf der ATC-Level-1-Platine zu steuern.

Am Ende kannst du:

- Tasterzustände auslesen
- LEDs direkt über Taster schalten
- einfache Logiken und Zustände umsetzen
- (Bonus) Blinkmuster starten und stoppen, ohne dass das Programm blockiert

3. Was ist auf der ATC-Platine bereits vorbereitet?

Auf der ATC-Level-1-Platine sind folgende Bauteile angeschlossen:

- **LED 1** ist an **Pin D5**
- **LED 2** ist an **Pin D6**
- **Taster 1** ist an **Pin D7**
- **Taster 2** ist an **Pin D8**

Du musst in dieser Übung nichts verdrahten.

Die Taster sind so ausgelegt, dass sie mit der internen Pull-Up-Schaltung des Arduino verwendet werden.

4. Neue Befehle/Konzepte in dieser Übung

In dieser Übung kommen erstmals digitale Eingänge hinzu.

Neue Befehle sind:

- `digitalRead()`
- `INPUT_PULLUP`
- logische Entscheidungen mit `if / else`

Hinweis:

Bei Verwendung von `INPUT_PULLUP` gilt:

- Taster nicht gedrückt → HIGH
- Taster gedrückt → LOW

5. Erklärung der verwendeten Befehle

`pinMode(pin, INPUT_PULLUP)`

Legt einen Pin als Eingang fest und aktiviert einen internen Widerstand. Dadurch wird kein externer Widerstand benötigt.

- nicht gedrückt → HIGH
- gedrückt → LOW

`digitalRead(pin)`

Liest den aktuellen Zustand eines Eingangs:

- HIGH oder LOW

`bool`

Ein Datentyp mit nur zwei Zuständen:

- `true`
- `false`

Ideal für Taster, Zustände und Entscheidungen.

`!` (logische Negation)

Kehrt einen logischen Wert um:

- `!true` → `false`
- `!false` → `true`

Wird verwendet um: aus „gedrückt = LOW“ einfach „gedrückt = `true`“ zu machen.

if (Bedingung)

Führt Code nur dann aus, wenn die Bedingung erfüllt ist.

Beispiel:

```
if (tasteGedrueckt) {  
    // Code wird nur hier ausgeführt  
}
```

millis()

Liefert die Zeit in Millisekunden seit Programmstart.

Im Gegensatz zu delay() hält diese Funktion das Programm nicht an.

unsigned long

Ein Datentyp für große positive Zahlen.

Wird für Zeitwerte in Kombination mit millis() verwendet.

6. Schritt-für-Schritt: Schreibe den Code selbst

6.1. Schritt 1 – Pins festlegen

Erstelle einen neuen Sketch und lege folgende Pins als Konstanten oder Variablen fest:

- LED 1
- LED 2
- Taster 1
- Taster 2

Aufgabe:

Nutze die Pin-Nummern 5, 6, 7 und 8.

Selbstcheck:

Du hast jetzt vier klar benannte Pins im Code.

6.2. Schritt 2 – setup(): Ein- und Ausgänge konfigurieren

In setup() musst du:

- die beiden LED-Pins als OUTPUT
- die beiden Taster-Pins als INPUT_PULLUP

konfigurieren.

Selbstcheck:

Im setup() stehen jetzt vier pinMode()-Aufrufe.

6.3. Schritt 3 – Aufgabe A: Taster steuert LED direkt

Jetzt reagiert das Programm direkt auf Benutzereingaben.

Aufgabe:

- Taster 1 gedrückt → LED 1 an
- Taster 1 losgelassen → LED 1 aus
- Taster 2 gedrückt → LED 2 an
- Taster 2 losgelassen → LED 2 aus

Hinweis:

Denke daran, dass ein gedrückter Taster bei INPUT_PULLUP den Wert LOW liefert.

Selbstcheck:

Die LEDs reagieren sofort und nur solange der jeweilige Taster gedrückt wird.

6.4. Schritt 4 – Aufgabe B: Taster als Schaltbefehl

Jetzt sollen die Taster Zustände ändern, nicht nur direkt reagieren.

Aufgabe:

- Taster 1 → beide LEDs an
- Taster 2 → beide LEDs aus

Die LEDs behalten ihren Zustand auch nach dem Loslassen der Taster.

Selbstcheck:

Ein kurzer Tastendruck reicht aus, um den Zustand zu ändern.

6.5. Schritt 5 – Aufgabe C: Umschaltlogik

Jetzt wird die Logik erweitert.

Aufgabe:

- Taster 1 → LED 1 an, LED 2 aus
- Taster 2 → LED 2 an, LED 1 aus

Selbstcheck:

Es ist immer nur eine LED aktiv.

6.6. Schritt 6 – Blinkmuster starten und stoppen (millis())

Jetzt kommt eine Aufgabe, bei der `delay()` an seine Grenze stößt.

Ziel:

- Taster 1 startet ein abwechselndes Blinkmuster
- Taster 2 beendet das Blinken
- Die Taster müssen jederzeit reagieren

Wichtig:

Mit `delay()` reagieren die Taster verzögert oder gar nicht.

In dieser Aufgabe wird erstmals eine nicht blockierende Zeitsteuerung verwendet. Statt das Programm mit `delay()` anzuhalten, wird die vergangene Zeit mit `millis()` abgefragt und darauf reagiert.

Grundidee von `millis()`

`millis()` liefert die Anzahl der Millisekunden, die seit dem Start des Programms vergangen sind.

Wichtig:

- `millis()` **wartet nicht**
- das Programm läuft ständig weiter
- Entscheidungen werden anhand von Zeitvergleichen getroffen

Das bedeutet:

Das Programm fragt nicht: „*Warte 500 ms*“, sondern: „*Sind 500 ms vergangen?*“

Prinzip

1. Die aktuelle Zeit wird gespeichert
2. In jedem Durchlauf von `loop()` wird geprüft:
 - Wie viel Zeit ist vergangen?
3. Ist der gewünschte Zeitraum überschritten:
 - wird der LED-Zustand geändert
 - wird die aktuelle Zeit neu gespeichert

Beispiel: Abwechselndes Blinken mit millis()

Hinweis:

Dieses Beispiel zeigt nur das Grundprinzip.
Es ist bewusst übersichtlich gehalten.

```
// Zeitsteuerung
unsigned long letzteZeit = 0;
const unsigned long intervall = 500;

// Blink-Zustand
bool ledZustand = false;

void loop() {
    unsigned long aktuelleZeit = millis();

    if (aktuelleZeit - letzteZeit >= intervall) {
        letzteZeit = aktuelleZeit;

        ledZustand = !ledZustand;

        digitalWrite(LED1_PIN, ledZustand);
        digitalWrite(LED2_PIN, !ledZustand);
    }
}
```

Erklärung zum Beispiel

- letzteZeit merkt sich, **wann zuletzt geschaltet wurde**
- intervall bestimmt die Blinkgeschwindigkeit
- millis() wird **nur gelesen**, nicht angehalten
- der LED-Zustand wird **umgeschaltet**, nicht neu berechnet

Während dieses Codes:

- laufen die LEDs weiter
- können Taster jederzeit abgefragt werden
- bleibt das Programm reaktionsfähig

Genau das ist mit delay() nicht möglich.

Merksatz zu millis()

millis() stoppt das Programm nicht.

Es ermöglicht, Zeit zu messen, während der Code weiterläuft.

Aufgabe (Hinweise, keine Lösung):

- Merke dir die aktuelle Zeit
- vergleiche sie mit einer gespeicherten Zeit
- schalte die LEDs nur, wenn die Zeit abgelaufen ist

Selbstcheck:

Das Blinkmuster läuft und kann jederzeit gestartet oder gestoppt werden.

7. Challenge

Diese Aufgaben bauen auf der Bonusaufgabe auf und erfordern saubere Logik. Sie sind bewusst anspruchsvoll.

7.1. Challenge A – Blinkrhythmus

Aufgabe:

- Taster 1 wählt Blinkrhythmus 1
- Taster 2 wählt Blinkrhythmus 2

Beispiel:

- Rhythmus 1 → langsames Blinken (z.B. 500 ms)
- Rhythmus 2 → schnelles Blinken (z.B. 200 ms)

Anforderung:

- Der gewählte Rhythmus bleibt aktiv, bis ein anderer ausgewählt wird
- Es darf immer nur ein Rhythmus aktiv sein

7.2. Challenge B – Geschwindigkeit verändern und LEDs ausschalten

Anfangs blinken beide LEDs mit 500ms

Aufgabe:

- Taster 1 erhöht Blinkgeschwindigkeit
- Taster 2 verringert Blinkgeschwindigkeit
- Taster 1 und Taster 2 gleichzeitig betätigt stoppt das Blinken (LEDs aus)

Hilfestellung – Blinkgeschwindigkeit verändern

Um die Blinkgeschwindigkeit zu verändern, muss ein Zahlenwert im Programm angepasst werden.

Dieser Wert bestimmt, wie viel Zeit zwischen zwei Blinkwechseln vergeht.

Statt einen festen Wert zu verwenden, kannst du diesen Wert vergrößern oder verkleinern.

Grundidee

- ein **größerer Zahlenwert** → langsames Blinken
- ein **kleinerer Zahlenwert** → schnelleres Blinken

Der Arduino kann mit Zahlen **rechnen**, genau wie ein Taschenrechner.

Beispielprinzip (ohne vollständigen Code)

Angenommen, du hast eine Variable für die Blinkzeit:

```
unsigned long blinkZeit = 500;
```

Wenn du möchtest, dass das Blinken langsamer wird, kannst du den Wert vergrößern:

```
blinkZeit = blinkZeit + 100;
```

Wenn du möchtest, dass das Blinken schneller wird, kannst du den Wert verkleinern:

```
blinkZeit = blinkZeit - 100;
```

Anwendung in der Challenge

- Taster 1 gedrückt: Blinkzeit vergrößern (langsamer)
- Taster 2 gedrückt: Blinkzeit verkleinern (schneller)

Der aktuelle Wert wird dabei gespeichert und beim nächsten Blinkvorgang automatisch verwendet.

8. Typische Fehler & schnelle Diagnose

- **LEDs reagieren gar nicht:**
Prüfe pinMode() und Pin-Nummern
- **Taster reagiert „verkehrt herum“:**
Prüfe, ob du LOW als „gedrückt“ behandelst.
- **Blinken stoppt nicht:**
Prüfe, ob dein Code blockierende delay()-Aufrufe enthält.

9. Referenzlösung Aufgabe A

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe A (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1

const int BTN2_PIN = 8; // Taster 2

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit interner Pull-Up-Schaltung

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

}

void loop() {

 // Taster einlesen

 // digitalRead() liefert HIGH oder LOW

 bool btn1Pressed = digitalRead(BTN1_PIN);

 bool btn2Pressed = digitalRead(BTN2_PIN);

 // Durch die logische Negierung (!) wird der Wert umgedreht:

 // HIGH (nicht gedrückt) -> LED aus

 // LOW (gedrückt) -> LED an

 digitalWrite(LED1_PIN, !btn1Pressed);

 digitalWrite(LED2_PIN, !btn2Pressed);

}

10. Referenzlösung Aufgabe B

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe B (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1: LEDs EIN

const int BTN2_PIN = 8; // Taster 2: LEDs AUS

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit interner Pull-Up-Schaltung

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

}

void loop() {

 // Aktuellen Zustand der Taster einlesen

 bool btn1 = digitalRead(BTN1_PIN);

 bool btn2 = digitalRead(BTN2_PIN);

 // Wenn Taster 1 gedrückt ist (LOW),

 // werden beide LEDs eingeschaltet

 if (btn1 == LOW) {

 digitalWrite(LED1_PIN, HIGH);

 digitalWrite(LED2_PIN, HIGH);

 }

 // Wenn Taster 2 gedrückt ist (LOW),

 // werden beide LEDs ausgeschaltet

 if (btn2 == LOW) {

 digitalWrite(LED1_PIN, LOW);

 digitalWrite(LED2_PIN, LOW);

 }

}

11. Referenzlösung Aufgabe C

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Aufgabe C (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1

const int BTN2_PIN = 8; // Taster 2

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit interner Pull-Up-Schaltung

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

}

void loop() {

 // Aktuellen Zustand der Taster einlesen

 bool btn1 = digitalRead(BTN1_PIN);

 bool btn2 = digitalRead(BTN2_PIN);

 // Wird Taster 1 gedrückt (LOW),

 // schaltet das Programm LED 1 ein

 // und LED 2 aus

 if (btn1 == LOW) {

 digitalWrite(LED1_PIN, HIGH);

 digitalWrite(LED2_PIN, LOW);

 }

 // Wird Taster 2 gedrückt (LOW),

 // schaltet das Programm LED 2 ein

 // und LED 1 aus

 if (btn2 == LOW) {

 digitalWrite(LED1_PIN, LOW);

 digitalWrite(LED2_PIN, HIGH);

 }

}

12. Referenzlösung Bonus

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Bonus (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1: Start

const int BTN2_PIN = 8; // Taster 2: Stop

// Merker, ob das Blinkmuster aktiv ist

bool blinking = false;

// Speichert den Zeitpunkt des letzten Umschaltens

unsigned long lastTime = 0;

// Zeitintervall für das Blinken (in Millisekunden)

const unsigned long intervalMs = 500;

// Speichert den aktuellen Blinkzustand:

// false -> LED 1 aus, LED 2 an

// true -> LED 1 an, LED 2 aus

bool blinkZustand = false;

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit interner Pull-Up-Schaltung

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

}

void loop() {

 // Aktuellen Zustand der Taster einlesen

 bool btn1 = digitalRead(BTN1_PIN);

 bool btn2 = digitalRead(BTN2_PIN);

 // Wird Taster 1 gedrückt, startet das Blinkmuster

 if (btn1 == LOW) {

 blinking = true;

 // Startzeit setzen, damit das Blinken sauber beginnt



```
    lastTime = millis();  
}  
  
// Wird Taster 2 gedrückt, wird das Blinkmuster gestoppt  
if (btn2 == LOW) {  
    blinking = false;  
  
    // LEDs ausschalten  
    digitalWrite(LED1_PIN, LOW);  
    digitalWrite(LED2_PIN, LOW);  
}  
  
// Blinken ohne delay():  
// Der Code läuft weiter, während die Zeit überprüft wird  
if (blinking) {  
  
    // Prüfen, ob das Zeitintervall abgelaufen ist  
    if (millis() - lastTime >= intervalMs) {  
  
        // Aktuellen Zeitpunkt speichern  
        lastTime = millis();  
  
        // Blinkzustand umschalten:  
        // aus -> an  
        // an -> aus  
        blinkZustand = !blinkZustand;  
  
        // LEDs entsprechend des Blinkzustands schalten  
        digitalWrite(LED1_PIN, blinkZustand);  
        digitalWrite(LED2_PIN, !blinkZustand);  
    }  
}  
}
```

13. Referenzlösung Challenge A

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Challenge A (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1: 500 ms

const int BTN2_PIN = 8; // Taster 2: 200 ms

// Speichert den Zeitpunkt des letzten Umschaltens

unsigned long lastTime = 0;

// Speichert das aktuelle Blinkintervall (in ms)

unsigned long intervalMs = 500; // Startwert: 500 ms

// Speichert den aktuellen Blinkzustand:

// false -> LED 1 aus, LED 2 an

// true -> LED 1 an, LED 2 aus

bool blinkZustand = false;

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit Pull-Up

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

}

void loop() {

 // Tasterzustände einlesen

 bool btn1 = digitalRead(BTN1_PIN);

 bool btn2 = digitalRead(BTN2_PIN);

 // Wenn Taster 1 gedrückt wird, wird das Intervall auf 500 ms gesetzt

 if (btn1 == LOW) {

 intervalMs = 500;

 }

 // Wenn Taster 2 gedrückt wird, wird das Intervall auf 200 ms gesetzt

 if (btn2 == LOW) {

 intervalMs = 200;

 }

 // Nicht blockierendes Blinken mit millis()



```
// Prüfen, ob genug Zeit vergangen ist
if (millis() - lastTime >= intervalMs) {
    lastTime = millis();      // Zeitpunkt merken
    blinkZustand = !blinkZustand; // Blinkzustand umschalten

    // LEDs abwechselnd schalten
    digitalWrite(LED1_PIN, blinkZustand);
    digitalWrite(LED2_PIN, !blinkZustand);
}
}
```

14. Referenzlösung Challenge B

Hinweis: Schau erst hier rein, wenn du es wirklich nicht gelöst bekommst.

// ATC Level 1 – Programmierübung 1.3 Challenge B (Referenzlösung)

// Pinbelegung der ATC-Platine

const int LED1_PIN = 5; // LED 1

const int LED2_PIN = 6; // LED 2

const int BTN1_PIN = 7; // Taster 1: schneller

const int BTN2_PIN = 8; // Taster 2: langsamer

// Merker, ob das Blinkmuster aktiv ist

bool blinking = true; // Start: Blinken läuft sofort

// Speichert den Zeitpunkt des letzten Umschaltens (für millis-Zeitmessung)

unsigned long lastTime = 0;

// Blinkintervall (in Millisekunden)

unsigned long intervalMs = 500; // Startwert: 500 ms

// Schrittweite für schneller/langsamer

const unsigned long STEP = 50;

// Speichert den aktuellen Blinkzustand:

// false -> LED 1 aus, LED 2 an

// true -> LED 1 an, LED 2 aus

bool blinkZustand = false;

// Entprell-/Sperr-Variablen:

// 0 = Taste ist "frei" (neuer Tastendruck kann gezählt werden)

// 1 = Taste ist "gesperrt" (Tastendruck wurde bereits gezählt)

int prell1 = 0;

int prell2 = 0;

void setup() {

 // LEDs als Ausgänge konfigurieren

 pinMode(LED1_PIN, OUTPUT);

 pinMode(LED2_PIN, OUTPUT);

 // Taster als Eingänge mit interner Pull-Up-Schaltung

 // nicht gedrückt -> HIGH

 // gedrückt -> LOW

 pinMode(BTN1_PIN, INPUT_PULLUP);

 pinMode(BTN2_PIN, INPUT_PULLUP);

 // Startzustand: beide LEDs aus

 digitalWrite(LED1_PIN, LOW);



```
digitalWrite(LED2_PIN, LOW);
}

void loop() {
    // Tasterzustände einlesen
    bool btn1 = digitalRead(BTN1_PIN);
    bool btn2 = digitalRead(BTN2_PIN);

    // Wenn beide Taster gleichzeitig gedrückt werden, wird das Blinken gestoppt
    if (btn1 == LOW && btn2 == LOW) {
        blinking = false;

        // LEDs ausschalten
        digitalWrite(LED1_PIN, LOW);
        digitalWrite(LED2_PIN, LOW);
    }

    // Wenn das Blinken gestoppt ist:
    // Ein Tastendruck startet das Blinken wieder
    if (!blinking) {
        if (btn1 == LOW || btn2 == LOW) {
            blinking = true;
            lastTime = millis();    // Zeitmessung neu starten
            blinkZustand = false;   // Blinkzustand zurücksetzen
        }
    }

    // Geschwindigkeit nur ändern, wenn das Blinken aktiv ist
    if (blinking) {

        // Taster 1 gedrückt -> schneller -> Intervall kleiner
        // prell1 sorgt dafür, dass pro Tastendruck nur EIN Schritt gezählt wird
        if (prell1 == 0 && btn1 == LOW) {
            intervalMs = intervalMs - STEP; // schneller: weniger Zeit zwischen Umschalten
            prell1 = 1;                      // sperren, damit nicht mehrfach gezählt wird
        }

        // Sobald Taster 1 losgelassen wird (HIGH), wird die Sperre wieder freigegeben
        if (btn1 == HIGH) {
            prell1 = 0;
        }
    }
}
```

```
// Taster 2 gedrückt -> langsamer -> Intervall größer
// prell2 sorgt ebenfalls für "ein Schritt pro Tastendruck"
if (prell2 == 0 && btn2 == LOW) {
    intervalMs = intervalMs + STEP; // langsamer: mehr Zeit zwischen Umschalten
    prell2 = 1;                    // sperren
}

// Sobald Taster 2 losgelassen wird, Sperre freigeben
if (btn2 == HIGH) {
    prell2 = 0;
}
}

// Nicht blockierendes Blinken (ohne delay)
if (blinking) {

    // Prüfen, ob das Blinkintervall abgelaufen ist
    if (millis() - lastTime >= intervalMs) {
        lastTime = millis();           // Zeitpunkt merken
        blinkZustand = !blinkZustand;  // Blinkzustand umschalten

        // LEDs abwechselnd schalten
        digitalWrite(LED1_PIN, blinkZustand);
        digitalWrite(LED2_PIN, !blinkZustand);
    }
}
}
```

15. Abschluss

Diese Programmierübung markiert den Übergang von einfachen Ausgaben zu echter Programmlogik.

Du hast gelernt, wie Eingaben, Zustände und zeitgesteuerte Abläufe zusammenspielen.

Damit bist du optimal auf komplexere Aufgaben vorbereitet.